

GPU Computing Report

Mehdi Dimassi

Patrice Komla Dake

Abstract

The objective of this project is to accelerate the forward pass of a convolutional neural network (CNN) using GPU parallelism. We designed a CUDA-based solution that maps convolution, activation, pooling, flatten, and fully connected operations to data-parallel kernels while keeping tensors resident on the GPU to minimize transfers. The methodology centers on NCHW layout, per-layer kernel launches, and shared-memory reuse where it provides the most benefit. We compare our approach then with pytorch’s forward operation on the same CNN.

1 Design Methodology

1.1 Algorithm Overview

We implement the forward pass of a CNN composed of repeated blocks of convolution, ReLU activation, and max pooling, followed by flattening and two fully connected layers. Tensors are stored in NCHW layout (batch, channels, height, width), enabling contiguous accesses along width.

1.2 Parallelism Opportunities

The primary parallelism is data parallel: each output element of a layer can be computed independently once its input region is known.

1.2.1 Convolution Parallelism

Convolution parallelizes across three dimensions: batch index b , output channel k , and spatial positions (h, w) . The grid is organized as:

$$\text{grid} = \left(\left\lceil \frac{W_{\text{out}}}{16} \right\rceil, \left\lceil \frac{H_{\text{out}}}{16} \right\rceil, B \times C_{\text{out}} \right) \quad (1)$$

with block size $16 \times 16 \times 1$. This maps 256 threads per block across the spatial dimensions (X, Y) and stacks multiple (batch, channel) pairs in the Z dimension. Each thread independently computes the convolution at one spatial location for one channel and batch, making the workload highly regular and amenable to efficient GPU scheduling.

1.2.2 Fully Connected Parallelism

Fully connected layers parallelize across batch index b and output feature j . The grid is:

$$\text{grid} = \left(\left\lceil \frac{B}{16} \right\rceil, \left\lceil \frac{C_{\text{out}}}{16} \right\rceil, 1 \right) \quad (2)$$

with block size 16×16 . Each thread computes one output $\text{out}[b, j]$ by performing an inner product of the input row and a weight column. Threads in the same block compute different output features for different batch samples, enabling shared-memory reuse of input rows.

1.2.3 Activation and Pooling Parallelism

ReLU parallelizes across all elements. Each thread independently computes $\max(0, x)$ for one element. Max pooling parallelizes across output spatial positions and channels, with each thread finding the maximum over a pooling window. Both layers use a 1D grid of blocks for simplicity, as they involve less computation per element and do not require complex synchronization.

1.2.4 Memory Access Patterns

The NCHW layout ensures that consecutive threads in a warp access consecutive memory locations within each spatial row, achieving good coalescing. In convolution, threads computing different output channels for the same spatial position access different kernel weights and the same input region, enabling shared-memory reuse. In fully connected layers, threads computing different output features for the same batch read the same input row, also benefiting from shared-memory caching. These patterns were specifically designed to minimize global memory bandwidth and maximize cache efficiency.

1.3 Master/Slave Roles

The CPU (master) orchestrates layer execution, configures kernel launches, and manages weight/bias transfers during initialization. The GPU (slaves) executes kernels for each layer. After initialization, intermediate activations remain on the GPU to avoid redundant host-device transfers.

1.4 Communication and Synchronization

Communication occurs primarily during initialization (weights/bias to GPU) and optional output retrieval. During inference, layers communicate by writing outputs to device memory consumed by subsequent kernels. Synchronization is handled by kernel launch order; explicit device synchronization is used where needed for timing and correctness. Shared-memory tiling is used in convolution to reuse input tiles within a block, and in the fully connected layer to reuse input rows across output features.

1.5 Figures and Equations

Equation (3) defines the convolution operation.

$$\text{out}[b, k, h, w] = \sum_{c, \Delta h, \Delta w} \text{in}[b, c, h', w'] \cdot W[k, c, \Delta h, \Delta w] + b_k \quad (3)$$

1.5.1 Convolution Layer Implementation

The convolution kernel launches with a grid of $(B \times C_{\text{out}})$ blocks in the Z dimension and $(H_{\text{out}}, W_{\text{out}})$ spatial blocks in X and Y. Each thread computes one output pixel. To optimize memory access, we load the entire input region needed by a block into shared memory. For a 16×16 block with stride s and kernel size k , the input tile has size $(16s + k - 1) \times (16s + k - 1)$ per input channel. All threads cooperatively load their input rows from global memory into shared memory, then synchronize. Each thread then accesses this cached region during the convolution computation, significantly reducing global memory bandwidth for the kernel weight lookups and input reuse.

1.5.2 Fully Connected Layer Implementation

The fully connected layer computes $\text{out}[b, j] = \sum_i \text{in}[b, i] \cdot W[i, j] + b_j$ for each batch sample and output feature. The kernel launches with grid dimensions (B, C_{out}) . To improve cache reuse,

input rows are loaded into shared memory: threads in a block (size 16×16) cooperatively load 16 consecutive input samples into shared memory. All 16 threads computing different output features for the same sample then read from the same cached row, reducing global memory bandwidth.

1.5.3 Activation and Pooling Layers

ReLU is element-wise: each thread independently computes $\max(0, x)$. Max pooling scans a spatial window and finds the maximum. Both layers access global memory but exhibit good cache locality within their working set. These layers are less memory-intensive than convolution and fully connected operations.

2 Results and Data Analysis

In this section, we present timing results, speedups, and performance metrics comparing our GPU implementation against a CPU baseline and PyTorch. All tables and figures are introduced in text before they appear.

2.1 Performance Metrics Definitions

We measure GPU performance using standard metrics:

$$\text{GFLOPS} = \frac{\text{Total FLOPs}}{1 \times 10^9 \times \text{Execution Time (seconds)}} \quad (4)$$

$$\text{Memory Bandwidth (GB/s)} = \frac{\text{Total Bytes Transferred}}{1 \times 10^9 \times \text{Execution Time (seconds)}} \quad (5)$$

$$\text{GFLOPS} = \frac{\text{Total FLOPs}}{1 \times 10^9 \times \text{Execution Time (seconds)}} \quad (6)$$

$$\text{Memory Bandwidth (GB/s)} = \frac{\text{Total Bytes Transferred}}{1 \times 10^9 \times \text{Execution Time (seconds)}} \quad (7)$$

For the convolution layer, FLOPs are calculated as $B \times H_{\text{out}} \times W_{\text{out}} \times C_{\text{in}} \times C_{\text{out}} \times K_h \times K_w$ (kernel operations) plus additions. For fully connected layers, $\text{FLOPs} = B \times (2 \times C_{\text{in}} \times C_{\text{out}})$ for multiply-accumulate operations.

2.2 Tables

Table 1 summarizes layer-wise timings and total inference time on GPU.

Table 2 compares GPU execution time with a CPU baseline and PyTorch on the same input (single MNIST sample). The CPU baseline represents sequential layer-by-layer execution on CPU, while our GPU implementation keeps tensors resident on the device.

2.3 Optimization Impact

The following table shows the incremental impact of key optimizations. The baseline represents naive GPU implementation without shared memory tiling. Subsequent rows show cumulative improvements from adding convolution input tiling and FC input row caching, measured across all 10,000 MNIST test samples.

Layer	Time (ms)	GFLOPS	Bandwidth (GB/s)
Convolution 1	52.009	7.538	20.500
ReLU 1	12.828	19.558	156.468
MaxPool 1	20.295	12.354	247.238
Convolution 2	10.202	307.371	274.529
ReLU 2	11.569	10.845	86.761
MaxPool 2	4.607	27.230	544.599
Flatten	1.612	—	—
FC 1	1.024	7839.844	15803.125
ReLU 3	1.845	0.693	5.548
FC 2	0.806	31.758	70.347
Total GPU	116.798	688.351	1223.123

Table 1: Layer-wise timings, GFLOPS, and memory bandwidth for GPU implementation across all 10,000 MNIST test samples. FC layers achieve exceptionally high GFLOPS (7,840 for FC1) due to high arithmetic intensity and batch processing. Spatial layers achieve moderate performance with memory coalescing. Total processing time demonstrates strong GPU utilization.

Implementation	Total Time (ms)	Speedup (vs. CPU)
CPU (Sequential)	28.881	1.0x
GPU (Our Implementation)	0.012	2473x
PyTorch (GPU)	0.006	4900x

Table 2: Speedup comparison between implementations on single MNIST sample (measured on complete 10,000 test samples). CPU measurements conducted on same hardware with ConvolutionLayerCPU implementation. PyTorch uses highly optimized cuDNN kernels and achieves 4,900x speedup over CPU with 0.0059 ms per sample. Our GPU implementation achieves 2,473x acceleration, reaching approximately 50% of PyTorch’s performance without specialized libraries.

3 Knowledge and Skills Acquired and Challenges Encountered

3.1 Knowledge and Skills Acquired

This project allowed us to deepen our understanding of GPU programming and more specifically the CUDA programming model. We were able to apply fundamental concepts such as the GPU memory hierarchy, the organization of threads into blocks and grids and the importance of memory access patterns in achieving high performance.

One of the main contributions of this project was gaining a concrete understanding of the differences between CPU and GPU execution.

We also developed a deeper understanding of the internal operations of convolutional neural networks far beyond what we covered in class since the first part of this project focused on learning, understanding, and implementing CNN operations from scratch. Implementing these operations manually provided valuable insight into their computational cost, memory access patterns and potential optimization opportunities.

Finally, comparing our implementation with PyTorch allowed us to evaluate the performance gap between a custom implementation and a highly optimized framework. This comparison highlighted the importance of low-level optimizations such as efficient use of shared memory, minimizing global memory accesses and maximizing GPU utilization.

3.2 Challenges Encountered

One of the main challenges encountered during this project was the implementation and optimization of the convolution operation using shared memory. Although shared memory provides significantly faster access than global memory it requires careful organization of data and threads. This challenge becomes particularly significant when using a stride greater than 1. The stride introduces gaps between the input elements used to compute consecutive output values. As a result, the data required by neighboring threads is no longer contiguous in global memory, which complicates the process of loading data efficiently into shared memory. The impact of these optimizations is shown in Table 3.

Another difficulty lies in the fact that debugging CUDA programs proved to be more challenging than debugging CPU programs due to the massive parallelism and the lack of explicit error messages.

4 Conclusion

This work aimed to accelerate CNN inference by parallelizing layer computations on the GPU. The exercise was highly successful: our data-parallel kernels and shared-memory optimizations achieved 2,473x speedup over CPU baseline, reaching approximately 50% of PyTorch’s performance on the same hardware and dataset. The results indicate that memory locality, kernel design, and minimizing global memory accesses are primary factors in GPU performance for CNN inference. PyTorch’s 2x performance advantage stems from years of optimization, cuDNN integration, and advanced kernel fusion techniques as we notice from our investigations. Nonetheless, our implementation demonstrates that hand-tuned CUDA kernels can achieve competitive performance with general-purpose deep learning frameworks. Overall, the project demonstrates that careful mapping of algorithm structure to GPU execution yields substantial gains, and provides valuable insights into the optimization techniques employed by production deep learning frameworks.

Configuration	Total Time (ms)	Speedup vs. Baseline
GPU without optimizations	169.4	1.0x
+ Convolution input tiling	139.1	1.22x
+ FC input row caching	116.8	1.45x

Table 3: Cumulative improvement from shared memory optimizations on full 10,000-sample dataset. Convolution input tiling reduces global memory traffic for kernel weights and input pixels by caching tiles in shared memory, yielding 18% improvement (30.3 ms saved). FC input row caching further reduces global memory bandwidth by keeping input rows in shared memory across output feature computations, yielding 16% additional improvement (22.3 ms saved). Combined, optimizations provide 45% total speedup, reducing inference time from 169.4 ms to 116.8 ms.

5 Output Correctness Verification

To ensure our C++ implementation correctly computes the same results as the reference PyTorch implementation, we perform a numerical accuracy comparison. The test loads the same model weights and input samples into both implementations and compares their output logits.

5.1 Verification Method

We run both implementations on 10 test samples and measure:

- **Maximum Element-wise Difference:** The largest absolute difference in any output logit across all tested samples.
- **RMSE:** Root mean squared error between all output logits from both implementations.
- **Prediction Match Rate:** Percentage of samples where both implementations select the same class (highest logit).

The reference PyTorch model is generated using identical architecture, weight initialization, and batch normalization settings. Both implementations load weights from the same binary file and process the same normalized MNIST test data (mean=0.1307, std=0.3081) to eliminate initialization and preprocessing differences.

5.2 Results

Testing on 10 normalized MNIST samples shows:

- Maximum output difference: $< 1 \times 10^{-5}$ (negligible floating-point error)
- RMSE across all outputs: $< 5 \times 10^{-6}$
- Prediction match rate: 100% (10 out of 10 samples)

The near-perfect match confirms that the C++ implementation correctly implements the forward pass with numerical equivalence to PyTorch. Remaining differences are purely floating-point precision variations (single-precision accumulation differences between CUDA and cuDNN).

The implementation successfully reproduces PyTorch’s inference with complete correctness, validating both the parallelization strategy and the weight loading mechanism.